request. The web url has the user id component which identifies the user object to update. The actual data is send in the body of the http request. Similar to create user API server accepts the User data as JSON and replies with updated user data in JSON.

Code can be found here: *http://pastebin.com/vveWBMrF*

The updateUser() method here uses *http.put()* method to update user on the server. First we need a distinct url to identify the specific user to update, In the updateUser() method, a new url is created by concatenating base user api and the id of a user. The user id is unique among users database and server can easily extract it from HTTP URL path.

Next lines of code are almost similar to *http.post()* usage, first parameter of *http.put()* is the web URL, second parameter is the JSON representation of user data and the third parameter is a RequestOptions object, it's purpose is same as previous createUser() method, to let server know the content type of request body, which is the JSON format.

If our request of user update is successful, the Response from server is converted back into User object and returned as a Javascript Promise to API consumer, else if the requests fail, catch function will convert it to rejected Promise.

Using Http Delete

Http delete is similar to Http Get request, the delete request does not contain request body, we could not send any data in body of delete requests, but url parameters can be used to pass data to server. The delete request is used to convey the removal of data.

We are using path parameter to convey which record is to be deleted by server. In following example a user is deleted by its own unique id. Server responds on successful delete operation with the deleted user's data as JSON in the response body.

Code can be found here: *http://pastebin.com/zMKmwZgk*

The `deleteUser` method takes the id of the user as input. The web api url is formed using the base user url and the id of user. The `http.delete()` method is then invoked with the web url and the request options.

Delete api returns deleted data as json string in response body. We parse it as user object using `res.json()` and returns the result as Promise. Errors are notified using the rejected Javascript promise object.